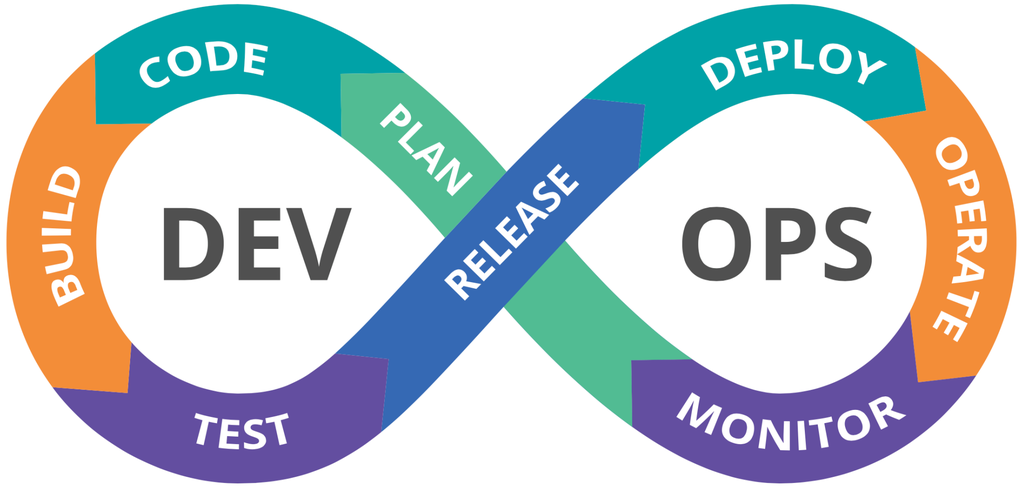
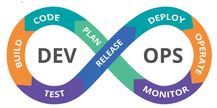



# Tehničke prakse kontinualnog učenja

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis




# Tehničke prakse kontinualnog učenja

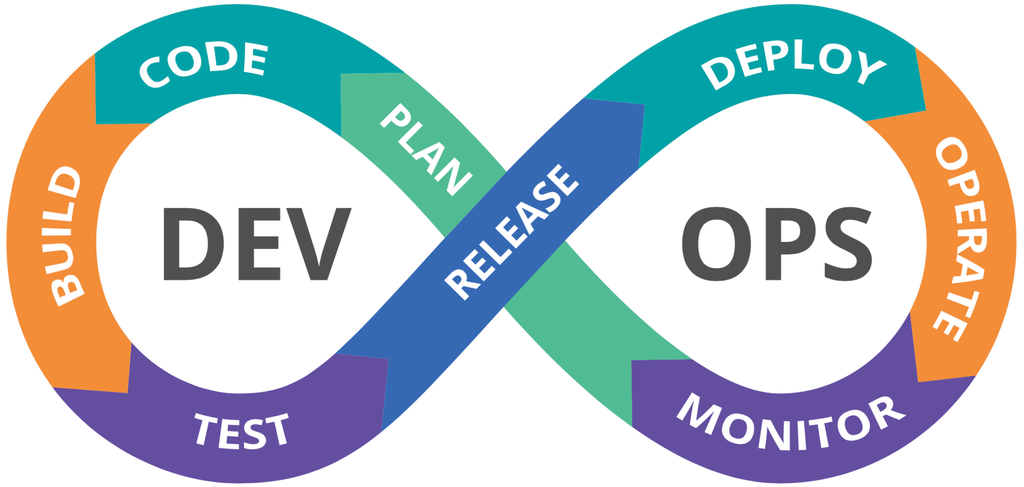
- Tehničke prakse koje kreiraju okruženje i mogućnosti za učenje, brzo, često, jeftino i na vreme. Ovo podrazumeva i učenje iz grešaka i incidenata, ali isto tako i organizovanje i razvoj sistema tako da se omogućava konstanto istraživanje i isprobavanje novih rešenja. Ovo podrazumeva:

  - Omogućavanje i ubacivanje učenja u svakodnevne zadatke - uspostavljanje odgovarajućeg okruženja za sigurnost

  - Injektovanje otkaza kako bi se kreirala otpornost sistema

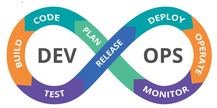
  - **Promocija lokalnih saznanja u globalna poboljšanja**

  - **Rezervisanje vremena za organizacioni razvoj i učenje**




# Tehničke prakse kontinualnog učenja
# Promocija lokalnih saznanja u globalna poboljšanja

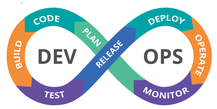
predavanja su bazirana i slike preuzete iz knjige
*"The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations"*
Gene Kim, Jez Humble, Patrick Debois, John Willis




# Promocija lokalnih saznanja u globalna poboljšanja

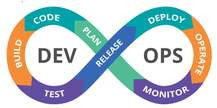
- Neophodno je kreirati mehanizme koji omogućavaju da se nova saznanja i unapređenja, koja su uočena lokalno, registruju, sačuvaju i podele na nivou cele organizacije.

- Na taj način postiže se multiplicirajući pozitivan efekat na akumulirano globalno znanje i unapređenja koja su implementirana. Svako iskustvo se koristi da se unapredi funkcionisanje u svim timovima - time unapređujući ukupno funkcionisanje organizacije.



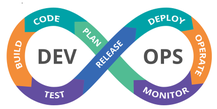

# Upotreba chatroom-a i chat botova za automatizaciju i registrovanje prikupljenog znanja

- Većina organizacija danas ima kreirane *chatroom-e* za timove.
- Ovo omogućava jednostavnu komunikaciju koja ostaje sačuvana i duže vreme
- Istovremeno, chatroomi se mogu iskoristiti i za okidanje automatskih radnji u development procesu
  - ovu tehniku je prvi koristio GitHub



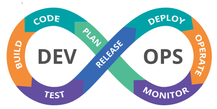

# Upotreba chatroom-a i chat botova za automatizaciju i registrovanje prikupljenog znanja

- Ideja je bila da se omogući da se iz samih razgovora pokrenu određene operacije
  - *Hubot* - softver koji je u vidu chat bota imao interakciju sa članovima ops tima
  - Pokretanje automatskih operacija je moguće zdavanjem komandi (u vidu poruka ciljanom chatbotu) direktno iz chatroom-a
    - @hubot deploy xyz to production
    - povratne poruke (rezultate deploymenta) hubot takođe objavljuje u chatroomu




# Upotreba chatroom-a i chat botova za automatizaciju i registrovanje prikupljenog znanja

- Korišćenje chatroom-a za razmenu informacija i pokretanje automatizovanih operacija ima nekoliko prednosti u poređenju sa klasičnim izdavanjem komandi putem terminala ili razmenom poruka emailovima
  - Svi članovi chatroom-a (članovi tima) vide i potpunosti su svesni operacija koje se pokreću na ovaj način
  - Čak i neiskusni (novi) članovi tima mogu lako da shvate dnevni ritam rada prateći poruke u chatroomu
  - Članovi tima su mnogo spremniji da pitaju kada vide da i drugi slobodno postavljaju pitanja preko chat-a
  - Omogućava se brzo učenje svih članova tima
- Osim toga chatroom-i su standardno podešeni da čuvaju istoriju komunikacije, tako da su izuzetno zgodni za pretraživanje kada treba ponoviti određene radnje i celokupna istorija dostupna je nerestriktivno svim članovima

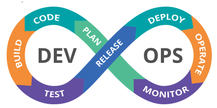



# Upotreba chatroom-a i chat botova za automatizaciju i registrovanje prikupljenog znanja

- GitHub je omogućio da Hubot trigeruje različite automatizovane alate (Puppet, Capistrano, Jenkins, resque, graphme…)

  - akcije koje hubot može da obavi obuhvataju proveru stanja (zdravlja servisa), puppet push operacije, deploymente na produkciona okruženja, isključivanje alarma za servise koji su prebačeni u offline zbog održavanja. Ponavljajuće akcije (prikupljanje logova smoke testova kada neki deployment ne prođe, isključivanje servera…)

- Komitovanje koda i komande koje izvršavaju izmene na produkcionim okruženjima takođe su podešeni tako da emituju poruke u chatroom. Na taj način svi imaju potpuni uvid u to što se dešava.



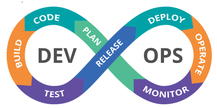

# Automatizacija i ponovno korišćenje standardnih softverskih procesa

- Često se standardi i procesi kako nešto radimo, koji se tiču arhitekture, testiranja, deploymenta, i upravljanja infrastrukturom zapisuju u vidu tekstualnih dokumenta.

- Problem je što dosta često, pogotovo novi članovi tima, ne znaju da ovi dokumenti postoje niti gde se nalaze ili kada i znaju nemaju dovoljno vremena da sve standarde primene.

  - Rezultat je da svako pravi neki sopstveni standard i proces kako obavlja određene zadatke.

  - Sve to dovodi do nestabilnog koda koji se teško održava i izvršnih okruženja koja su teška i skupa za eksploataciju

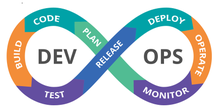



# Automatizacija i ponovno korišćenje standardnih softverskih procesa

- Umesto dokumentovanja standarda i procesa u tekstualnoj formi, neophodno je ovu dokumentaciju prevesti u formu koja je izvršiva u automatizovanim okruženjima i što omogućava da se ti standardi i procesi lako ponovo koriste.

- Pre svega - neophodno je prikupljeno znanje (automatizovne skripte i slično), čuvati na sistemima za verzioniranje koda (ovo je poželjno čak i za dokumentaciju u prostom tekstualnom obliku). Pri tome treba obezbediti alate koji omogućavaju da se ovi repozitorijumi koda lako pretražuju.

- Jednom kada se standardi i procesi koje sprovodimo zapišu u formi automatizovanih izvršivih skripti, tada ti standardi i procesi postaju ponovljivi i široko prihvaćeni, povećavajući stabilnost deployovanih rešenja.

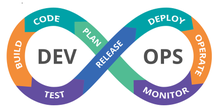



# Kreiranje jedinstvenog deljenog repozitorijuma koda za celu organizaciju

- Ovo je jedan od najjefikasnijih mehanizama za instant širenje saznanja kroz celu organizaciju.

- Kada se na ovom jedinstvenom repozitorijumu koda ažurira određeni deo koda (npr. neka deljena biblioteka koju koristi više timova), ova izmena je automatski vidljiva i biće implementirana (možda ne odmah) od strane svih timova.

- Google je već do 2015 uspostavio ovakav jedinstveni repozitorijum koda, sa preko milijardu fajlova, koji pokriva sva njihova softverska rešenja, Google Search, Google Maps, Google Docs, Google+, Google Calendar, Gmail, and YouTube i aktivno ga koristi preko 25000 inženjera

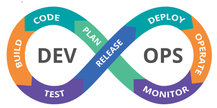



# Kreiranje jedinstvenog deljenog repozitorijuma koda za celu organizaciju

- Ovaj pristup je zapravo stvorio “riznicu” biblioteka koda, što je od velike vrednosti za svaki razvojni tim, jer postoje standardizovana rešenja za većinu stvari koja se pojavi kao potrebna u određenom projektu.

- Pri tome svakom inženjeru koji ima potrebu da iskoristi neki kod, taj kod je dostupan uvek u svojoj najnovijoj verziji

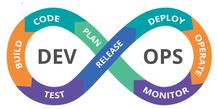
- Osim samog izvornog koda - na ovom repozitorijumu se čuvaju i svi drugi artefakti koji sadrže “produkciono znanje”
  - konfiguracioni standardi (Chef recipes, Puppet manifests…)
  - Deployment alati
  - Standardi za sprovođenje testova, alati za testiranje
  - Alati za deployment pipeline
  - Alati za monitoring i analizu
  - Tutorijali i standardi




# Kreiranje jedinstvenog deljenog repozitorijuma koda za celu organizaciju

- Osim što omogućava da svakome bude dostupna poslednja verzija bilo kog koda, i time obezbeđuje da svi imaju pristup poslednjim saznanjima o bilo kom problemu, ovaj jedinstveni repozitorijum je istovremeno i odličan mehanizam za sprečavanje masovnih otkaza deployovanih projekata.

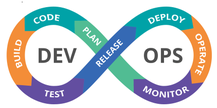
  - Svaki put kada se komituje neki kod, pokreće se bildovanje, koje uvek uzima najnovije verzije svega korišćenog - time se omogućava da se sve greške otkriju rano i da su svi kodovi ažurni i stabilni




# Kreiranje jedinstvenog deljenog repozitorijuma koda za celu organizaciju

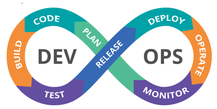
- U Google-ovoj implementaciji principa jedinstvenog repozitorijuma, svaka biblioteka koda ima svog “vlasnika” koji je zadužen da brine da se ona u svakom momentu kompajlira, već i da korektno prolazi testove u svim projektima u koje je uključena.

  - Vlasnik je istovremeo zadužen i da nadgleda proces migracije projekata na novije verzije “svoje” biblioteke. Ključno za uspešne migracije je postojanje automatizovanih testova.
  - U idealnoj situaciji samo jedan vlasnik i jedan tim održavaju određenu biblioteku, i pri tome je jako bitno da se usvoji princip da se dozvoljavaju bildovi drugih projekata samo sa jednom (poslednjom stabilnom) verzijom biblioteke. Vlasnik je odgovoran i za ispomoć drugim timovima tokom migracije na novu verziju.
    - Ovo automatski podrazumeva da moraju postojati dobri, automatizovani, regresioni testovi
  - Pogodno je i da postoje chatovi o određenim bibliotekama gde se mogu dobiti odgovori kako od samog razvojnog tima tako i od drugih timova koji koriste određenu biblioteku




# Širenje znanja korišćenjem automatizovanih testova i primenom usvojenih praksi

- Kada već postoji veliki broj deljenih biblioteka koje se koriste u raznim projektima neophodno je obezbediti da su ove biblioteke “pokrivene” odgovarajućim brojem automatizovanih testova.
  - Na ovaj način biblioteke postaju dokumentovane jer se iz testova može između ostalog razumeti i kako i u kom kontekstu se koristi kod biblioteke.
  - Ovo je posebno istaknuto ako se koristi TDD, gde se testovi pišu pre same implementacije koda.

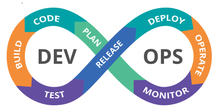



# Razmišljati o operativnim problemima tokom dizajniranja aplikacije

- Kada članovi razvojnog tima prate šta se dešava sa njihovim kodom u ostatku produkcionog lanca, pomažući pri tome prilikom rešavanja incidenata u produkcionim okruženjima, aplikacija vremenom postaje inherentno bolje dizajnirana - vodeći računa o problemima sa kojim se mogu susresti operativni timovi u eksploataciji.

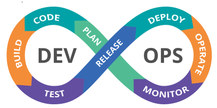
- Kada se prilikom dizajna same aplikacije vodi računa da kreirani kod omogućava brz protok kroz deployment pipeline i brzu isporuku na produkciona okruženja, vrlo verovatno se vremenom identifikuju nefunkcionalni zahtevi koje je neophodno integrisati u rešenje.

- Ovi nefunckionalni zahtevi upravo treba da omoguće brz deployment, stabilan rad u produkcionim okruženjima, laku detekciju anomalija, sve ono što i inače treba za rad operativnim timovima.




# Razmišljati o operativnim problemima tokom dizajniranja aplikacije

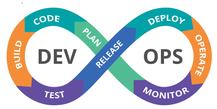
- Primer nefunkcionalnih zahteva o kojima treba povesti računa:
  - Dovoljno telemetrijskih podataka iz same aplikacije, ali i izvršnih okruženja
  - Mogućnost da se lako isprate zavisnosti između raznih modula
  - Dizajn servisa koji su “otporni” i degradiraju performanse umesto da potpuno otkažu u slučaju problema (*degrade gracefully*)
  - Kompatibilnost između verzija
  - Mogućnost arhiviranja podataka, kako bi se upravljalo veličinom skupova podataka nad kojima aplikacija manipuliše
  - Mogućnost da se logovi suvislo pretraže i izvuku korisne informacije
  - Mogućnost praćenja korisničkih zahteva dok oni tranzitiraju između različitih servisnih modula
  - Jednostavna, centralizovana konfiguraciona platforma za kod u produkciji, sa korišćenjem *feature flagova ( switcheva )*




# Dodavanje ponovljivih operativnih “korisničkih scenarija” u razvoj

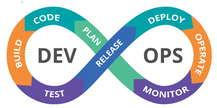
- Težimo ka tome da se što je moguće više operacija koje operativni timovi obavljaju automatizuje (skriptovima i alatima koji omogućavaju automatizovano upravljanje konfiguracijama)

- Kada postoje operacije koje se ne mogu automatizovati (npr. fizičko dodavanje novog servera u rack i njegovo povezivanje na mrežu, napajanje i sl.) trebalo bi težiti ka tome da se definišu jasni protokoli posebno u pogledu *handoff* tačaka (tačak u kome se zahtevi jednog tima predaju na izvršenje drugom timu), kako bi se smanjilo kašnjenje. Ovo omogućava da se lakše planira potrebno vreme, resursi za buduće ponavljanje sličnih operacija.




# Dodavanje ponovljivih operativnih “korisničkih scenarija” u razvoj

- Idealno, za svaki ponavljajući zadatak koji operativni timovi ponekad moraju da obave bi bilo dobro da se zna:
  - koji posao se treba obaviti
  - ko je sve neophodan da bi se posao obavio
  - koji su koraci do završetka datog posla
  - koliko ceo proces obično traje



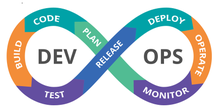

# Potvrditi da izbor tehnologija omogućava postizanje željenih ciljeva

- Jedan od ciljeva svake softverske firme jeste podići produktivnost razvojnih timova.

- Kod servisno orijentisanih arhitektura timovima se omogućava da rešenje izrade u onoj tehnologiji koja im se čini najprimerenija za rešenje problema.

  - Iako je ovaj princip vrlo produktivan, može dovesti do problema, kada se recimo celokupno tehnološko znanje o određenom kritičnom timu nalazi samo u jednom timu.

  - U tom slučaju samo taj tim je u stanju da dalje razvija, rešava probleme u tom servisnom modulu - potencijalno “usko grlo”. U tom slučaju se povećanje produktivnosti tima desilo nauštrb produktivnosti i efektivnosti cele organizacije.

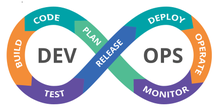



# Potvrditi da izbor tehnologija omogućava postizanje željenih ciljeva

- Ovo posebno može da postane problem ako članovi operativnih timova nemaju dovoljna znanja da se “izbore” sa datim rešenjem u produkcionom okruženju.

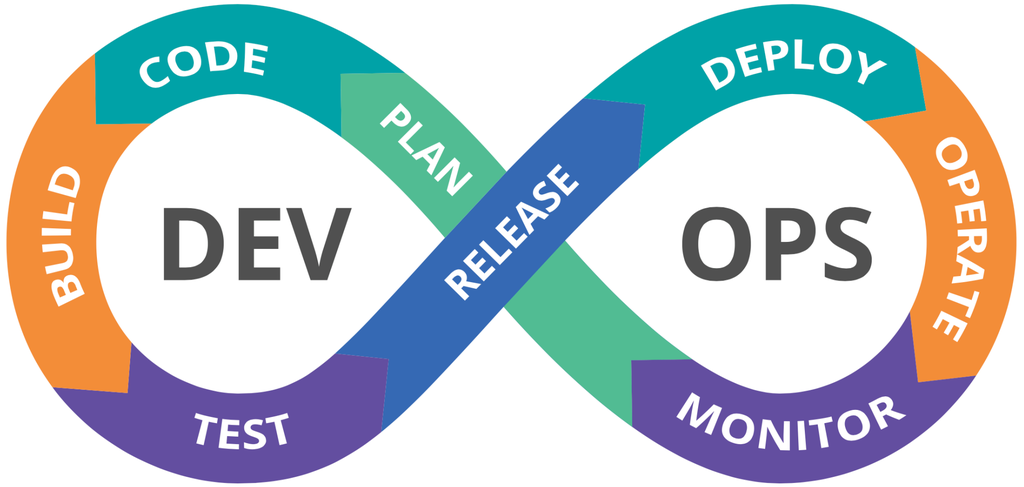
- Bitno je obezbediti da članovi operativnih timova mogu da iskažu i svoje mišljenje o tehnologiji kojom će određene komponente biti implementirane.

- Operativni timovi bi trebali da mogu da saopšte ili daju listu tehnoloških platformi za koje pružaju podršku - pri tome ta lista tehnologija bi trebalo da je nastala kao rezultat konsenzusa između razvojnih i operativnih timova (ne želimo da operativni timovi imperativno nameću tehnologiju koja “davi” dalji razvoj).



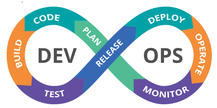

# Potvrditi da izbor tehnologija omogućava postizanje željenih ciljeva

- Kada se kreira lista podržanih tehnologija treba ih analizirati tako da se otkriju one koje stvaraju nesrazmerno veliki dodatni posao, ili neplanirani posao, da bi se rešenje implementiralo u produkcionim okruženjima:
  - tehnologije koje usporavaju protok radnih zadataka kroz celokupan razvojni proces
  - zahtevaju veliku količinu neplaniranog posla za puštanje u rad
  - kreiraju naknadno veliki broj zahteva za podršku
  - nekompatibilne su sa planiranom softverskom arhitekturom rešenja
- Uklanjanje tehnologija koje proizvode ove probleme, operativnim timovima se oslobađa vreme da se posvete izgradnji i održavanju infrastrukture koja omogućava pouzdan i stabilan rad naših rešenja u produkciji.




# Tehničke prakse kontinualnog učenja
# Rezervisanje vremena za organizacioni razvoj i učenje

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis




# Rezervisanje vremena za organizacioni razvoj i učenje

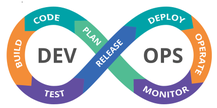
- Jedna od praksi koju je promovisala Toyota
  - improvement blitz (kaizen blitz)
  - vreme posebno posvećeno za diskusiju i analizu određenih problema, često u trajanju od nekoliko dana
  - cilj ovih okupljanja i razmene mišljenja je da se unapredi proces proizvodnje
    - često je rezultat *kaizen blitz*-a novi pristup rešavanju određenog problema (novi raspored opreme u proizvodnoj liniji, redosled operacija i sl.)
    - često se pravi i “*to do*” lista unapređenja koja treba naknadno sprovesti

# Rezervisanje vremena za organizacioni razvoj i učenje

- Implementacija u DevOps-u tzv. DevOps Dojo - praktični treninzi tokom kojih inženjeri imaju priliku da prođu kroz obuku kojom unapređuju svoje znanje

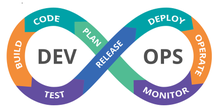
  - *30 days challenge*

  - razovojni timovi rade sa instruktorima prolazeći kroz sve faze razvoja - planiranje, rad na zadacima, demoi u dvodnevnim sprintovima




# Institucionalizacija određenih rituala kako bi se razrešili zaostali problemi (*technical debth*)

- rezerviše se određeno vreme koje se posvećuje samo uklanjanju postojećih bugova ili nezadovoljenih funkcionalnih ili nefunkcionalnih zahteva
  - jedan od najlakših načina da se uvede jednodnevni ili jednonedeljni “improvement blitz” - sprint.
    - takođe je moguće da se u planiranju sprintova uvede da je svaki 4 ili 5 sprint bez razvoja novih feature, dakle improvement sprint u kome se samo adresiraju postojeći nerešeni problemi
    - *spring cleanings, fall cleanings, ticket queue inversion week,*
  - *20% innovation time* - vreme rezervisano tako da se osim uklanjanja postojećih problema analizira upotreba novih tehnologija za njihovo rešavanje
    - hackaton - svako ponudi prototip nekog rešenja za određene probleme, na kraju se bira najefikasnije, najbolje rešenje za predstavljeni problem
- na kraju ovih vremenskih intervala svaki tim prezentuje napredak koji je napravio




# Omogućavanje svim zaposlenima da uče, ali i da prenose svoje znanje (predaje)

- dinamično radno okruženje podrazumeva da svaki član tima može da uči od drugih, ali i da svaki član tima koji je stekao neko vredno znanje može i ima mogućnost da to znanje prenese drugima

- čest način da se ovo obezbedi je da se rezerviše određeno vreme za ove potrebe (konferencije, workshop-i, mentorstva…)

  - npr. svakom članu tima se ostavlja 4 sata nedeljno da uči tehnologije, proces ili bilo šta drugo što smatra korisnim za svoj profesionalni razvoj

  - najvrednija stvar koju stariji članovi tima mogu pružiti juniorima je mentorstvo

- ključno je da svi članovi tima shvate da je pogotovo u visokotehnološkoj sferi proces učenja doživotan, te da to što u određenom momentu nešto ne zna ne treba da bude blokirajući momenat, već momenat koji ga motiviše za dalje učenje.

  - Svakodnevno se odeđene veštine mogu unapređivati i učestvovanjem u code review procesu



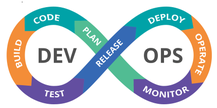

# Deljenje znanja stečenog na konferencijama

- Često je učešće na konferencijama ograničeno finansijskim resursima, i više se očekuje da članovi tima uče jedni od drugih

- Idealno bi bilo da sve organizacije rezervišu određena sredstva za svakog zaposlenog kako bi imao priliku da učestvuje na konferencijama i workshop-ima

  - Ali ne mogu svi ići na sve konferencije

  - Organizovati “interne konferencije”, “sesije učenja”, gde će oni koji su prisustvovali nekim konferencijama, workshopima, događajima i stekli određena vredna znanja preneti ta svoja iskustva ostalim zaposlenima




# Održavanje internih konsultacija i obuka

- Kako bi se saznanja koja određeni timovi ili članovi timova stiču tokom razvoja određenih softverskih rešenja prenela i korisno upotrebila u celoj organizaciji poželjno je da se održavaju interne konsultacije i interne obuke
  - Ove konsultacije mogu biti u bilo kojoj formi (vreme za razgovor, organizvoanje sesija za sve zainteresovane, neformalni sastanci na kojima se priča o rešenjima…)
  - Vrlo često za ovo nije potreban nikakav budžet (osim da menadžment shvati i prihvati da inženjeri neće sve sate nedeljno provoditi kodirajući)
    - Googleov TotT